In [1]:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Dropo
import keras
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# Load the dataset
df = pd.read_csv('Ansa_7thjune.csv')
#df = pd.read_csv('Ansa_7thjune_2nd_iteration.csv')
dataframe = pd.DataFrame(df)
print('data frame',df)

# Split the data into features (X) and labels (y)
x = dataframe.iloc[:,1:2].values
y = dataframe.iloc[:,2:3].values

#x = df.dropna(df[:1])
print('X values', x)
print('Y values', y)
print('shape of x',np.shape(x))
print('shape of y',np.shape(y))

# Split the data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_
print('x train', x_train)
print('y train', y_train)

```

```
data frame       Serial   emg  label
0           1  1501      0
1           2  1569      0
2           3  1552      0
3           4  1554      0
4           5  1527      0
...       ...   ...    ...
5495    5496  1680      0
5496    5497  1730      0
5497    5498  1694      0
5498    5499  1747      0
5499    5500  1685      0

[5500 rows x 3 columns]
X values [[1501]
 [1569]
 [1552]
 ...
 [1694]
 [1747]
 [1685]]
```

```
Y values [[0]
 [0]
 [0]
 ...
 [0]
 [0]
 [0]]
shape of x (5500, 1)
shape of y (5500, 1)
x train [[1907]
 [1845]
 [1695]
 ...
 [1904]
 [1721]
 [1662]]
y train [[0]
 [1]
 [0]
 ...
 [0]
 [0]
 [0]]
```

In [2]:

```
import tensorflow as tf

from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatt

# Define the model architecture

model = tf.keras.Sequential([

    tf.keras.layers.Conv1D(filters=32, kernel_size=3, activatio

    #tf.keras.layers.Conv1D(filters=64, kernel_size=3, activati

    #tf.keras.layers.Conv1D(filters=128, kernel_size=3, activat

    #tf.keras.layers.Conv1D(filters=256, kernel_size=3, activat

    tf.keras.layers.GlobalAveragePooling1D(data_format="channel

    tf.keras.layers.Dense(256, activation='relu'),

    tf.keras.layers.Dense(1, activation='sigmoid')

])
print(model.summary())

```

```
/Users/ranin/anaconda3/lib/python3.10/site-packages/keras/src/laye
rs/convolutional/base_conv.py:107: UserWarning: Do not pass an `in
put_shape`/`input_dim` argument to a layer. When using Sequential
```

```
models, prefer using an `Input(shape)` object as the first layer i
n the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kw
args)
```

Model: "sequential"

Layer (type)	Output Shape	
conv1d (Conv1D)	(None, 100, 32)	
global_average_pooling1d (GlobalAveragePooling1D)	(None, 32)	
dense (Dense)	(None, 256)	
dense_1 (Dense)	(None, 1)	

Total params: 8,833 (34.50 KB)

Trainable params: 8,833 (34.50 KB)

Non-trainable params: 0 (0.00 B)

None

In [4]:
```
# Compile the model

model.compile(optimizer='adam', loss='binary_crossentropy', met

# Train the model

history =  model.fit(x_train, y_train, validation_data=(x_test,

# write history into csv file
```

```
Epoch 1/500
69/69 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.5070 - loss:
13.2640 - val_accuracy: 0.4145 - val_loss: 0.7006
Epoch 2/500
69/69 ━━━━━━━━━━━━━━━━━━━━ 0s 757us/step - accuracy: 0.5164 - los
s: 0.7850 - val_accuracy: 0.4145 - val_loss: 1.6746
Epoch 3/500
69/69 ━━━━━━━━━━━━━━━━━━━━ 0s 779us/step - accuracy: 0.5077 - los
s: 2.2763 - val_accuracy: 0.5855 - val_loss: 1.3606
Epoch 4/500
69/69 ━━━━━━━━━━━━━━━━━━━━ 0s 813us/step - accuracy: 0.4969 - los
s: 1.2085 - val_accuracy: 0.5855 - val_loss: 0.6799
Epoch 5/500
69/69 ━━━━━━━━━━━━━━━━━━━━ 0s 884us/step - accuracy: 0.5034 - los
s: 0.8503 - val_accuracy: 0.5855 - val_loss: 1.2268
Epoch 6/500
69/69 ━━━━━━━━━━━━━━━━━━━━ 0s 841us/step - accuracy: 0.5085 - los
s: 1.0285 - val_accuracy: 0.5855 - val_loss: 0.6769
Epoch 7/500
```

In [5]:
```
import pandas as pd

# Save history to a CSV file
history_df = pd.DataFrame(history.history)
history_df.to_csv('history.csv')
```

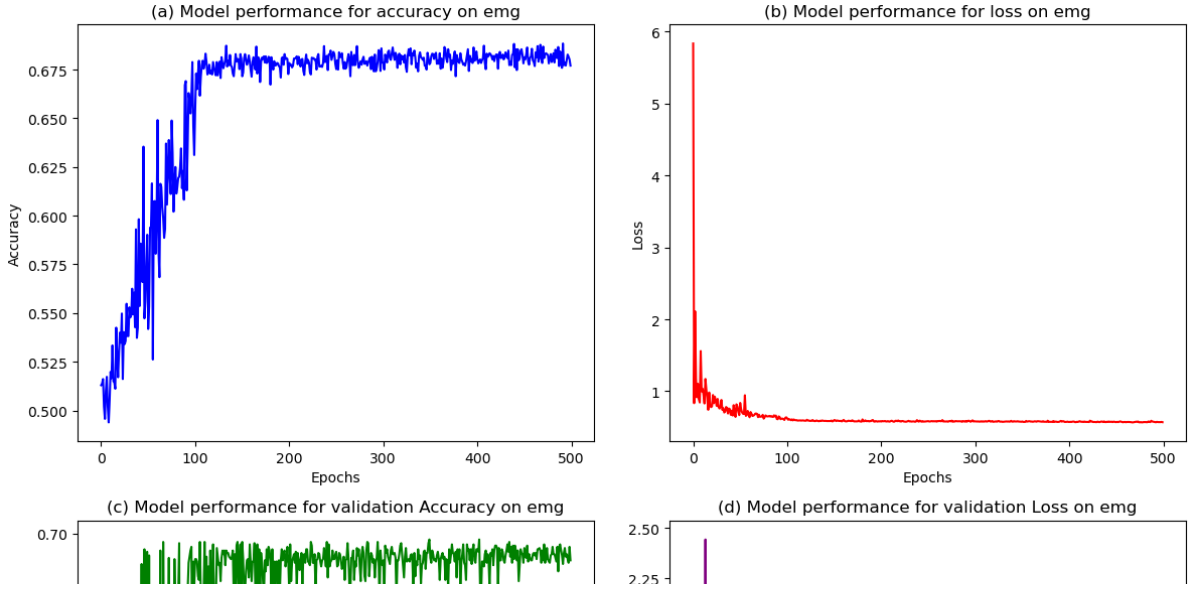
In [8]:
```
'''import pandas as pd
import matplotlib.pyplot as plt

# Load Data
df = pd.read_csv('history.csv')

# Plot Accuracy
plt.figure(figsize=(8, 6))
plt.plot(df['serial'], df['accuracy'], label='Accuracy', color=
plt.xlabel('Serial')
plt.ylabel('Accuracy')
plt.title('Model performance for accuracy')
plt.legend()
plt.grid(True)
plt.show()
```

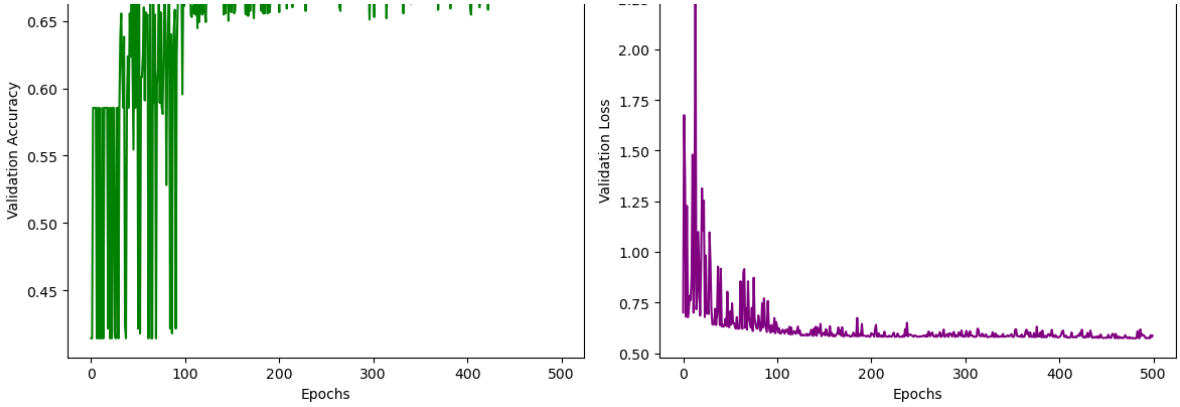
```
# Plot Loss
plt.figure(figsize=(8, 6))
plt.plot(df['serial'], df['loss'], label='Loss', color='red')
plt.xlabel('Serial')
plt.ylabel('Loss')
plt.title('Model performance for loss')
plt.legend()
plt.grid(True)
plt.show()

# Plot Validation Accuracy
plt.figure(figsize=(8, 6))
plt.plot(df['serial'], df['val_accuracy'], label='Validation Ac
plt.xlabel('Serial')
plt.ylabel('Validation Accuracy')
plt.title('Model performance for validation Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# Plot Validation Loss
plt.figure(figsize=(8, 6))
plt.plot(df['serial'], df['val_loss'], label='Validation Loss',
plt.xlabel('Serial')
plt.ylabel('Validation Loss')
plt.title('Model performance for validation Loss')
plt.legend()
plt.grid(True)
plt.show()
'''

```

Out[8]:
```
"import pandas as pd\nimport matplotlib.pyplot as plt\n\n# Load Da
ta\ndf = pd.read_csv('history.csv')\n\n# Plot Accuracy\nplt.figur
e(figsize=(8, 6))\nplt.plot(df['serial'], df['accuracy'], label='A
ccuracy', color='blue')\nplt.xlabel('Serial')\nplt.ylabel('Accurac
y')\nplt.title('Model performance for accuracy')\nplt.legend()\npl
t.grid(True)\nplt.show()\n\n# Plot Loss\nplt.figure(figsize=(8,
6))\nplt.plot(df['serial'], df['loss'], label='Loss', color='red')
\nplt.xlabel('Serial')\nplt.ylabel('Loss')\nplt.title('Model perfo
rmance for loss')\nplt.legend()\nplt.grid(True)\nplt.show()\n\n# P
lot Validation Accuracy\nplt.figure(figsize=(8, 6))\nplt.plot(df['
serial'], df['val_accuracy'], label='Validation Accuracy', color='
green')\nplt.xlabel('Serial')\nplt.ylabel('Validation Accuracy')\n
plt.title('Model performance for validation Accuracy')\nplt.legen
d()\nplt.grid(True)\nplt.show()\n\n# Plot Validation Loss\nplt.fig
ure(figsize=(8, 6))\nplt.plot(df['serial'], df['val_loss'], label
='Validation Loss', color='purple')\nplt.xlabel('Serial')\nplt.yla
bel('Validation Loss')\nplt.title('Model performance for validatio
n Loss')\nplt.legend()\nplt.grid(True)\nplt.show()\n"
```

In [6]:
```
import pandas as pd
import matplotlib.pyplot as plt

```

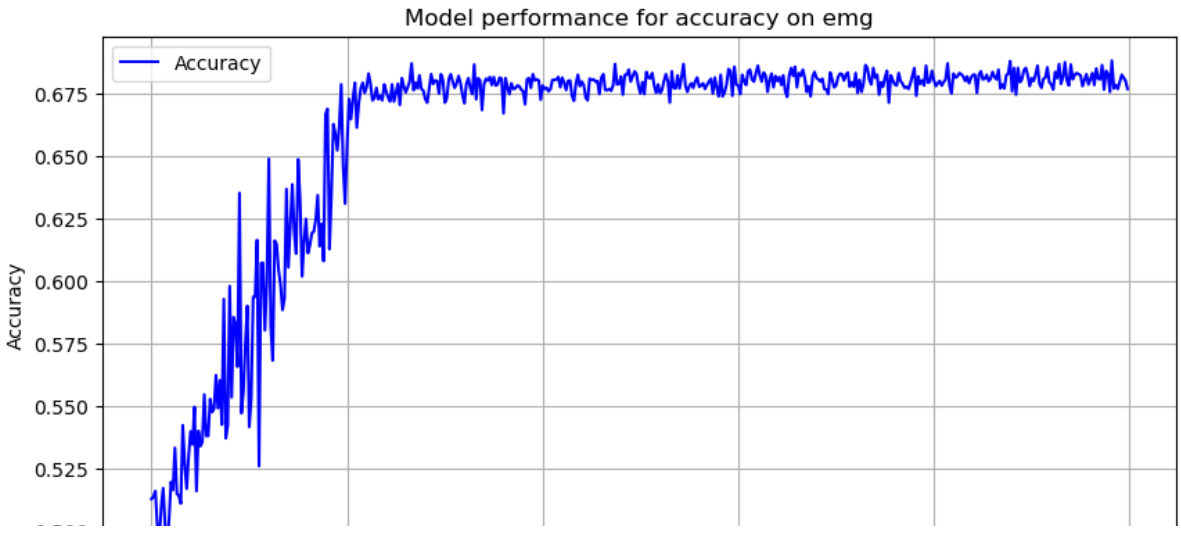
```
# Load Data
df = pd.read_csv('history.csv')

# Create a 2x2 subplot layout
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

# Plot Accuracy
axs[0, 0].plot(df['serial'], df['accuracy'], label='Accuracy',
axs[0, 0].set_xlabel('Epochs')
axs[0, 0].set_ylabel('Accuracy')
axs[0, 0].set_title('(a) Model performance for accuracy on emg'

# Plot Loss
axs[0, 1].plot(df['serial'], df['loss'], label='Loss', color='r
axs[0, 1].set_xlabel('Epochs')
axs[0, 1].set_ylabel('Loss')
axs[0, 1].set_title('(b) Model performance for loss on emg')

# Plot Validation Accuracy
axs[1, 0].plot(df['serial'], df['val_accuracy'], label='Validat
axs[1, 0].set_xlabel('Epochs')
axs[1, 0].set_ylabel('Validation Accuracy')
axs[1, 0].set_title('(c) Model performance for validation Accur

# Plot Validation Loss
axs[1, 1].plot(df['serial'], df['val_loss'], label='Validation
axs[1, 1].set_xlabel('Epochs')
axs[1, 1].set_ylabel('Validation Loss')
axs[1, 1].set_title('(d) Model performance for validation Loss

# Adjust layout
plt.tight_layout()

# Show plots
plt.show()

```

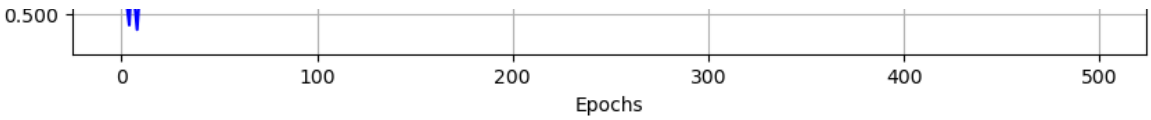


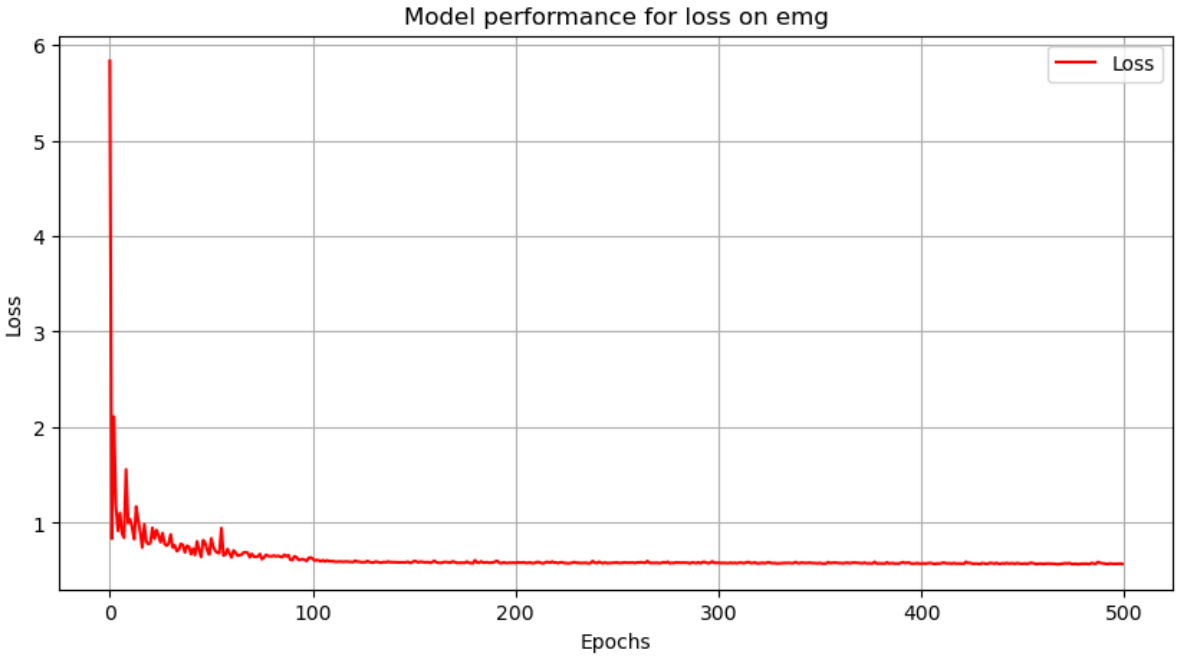


In [7]:

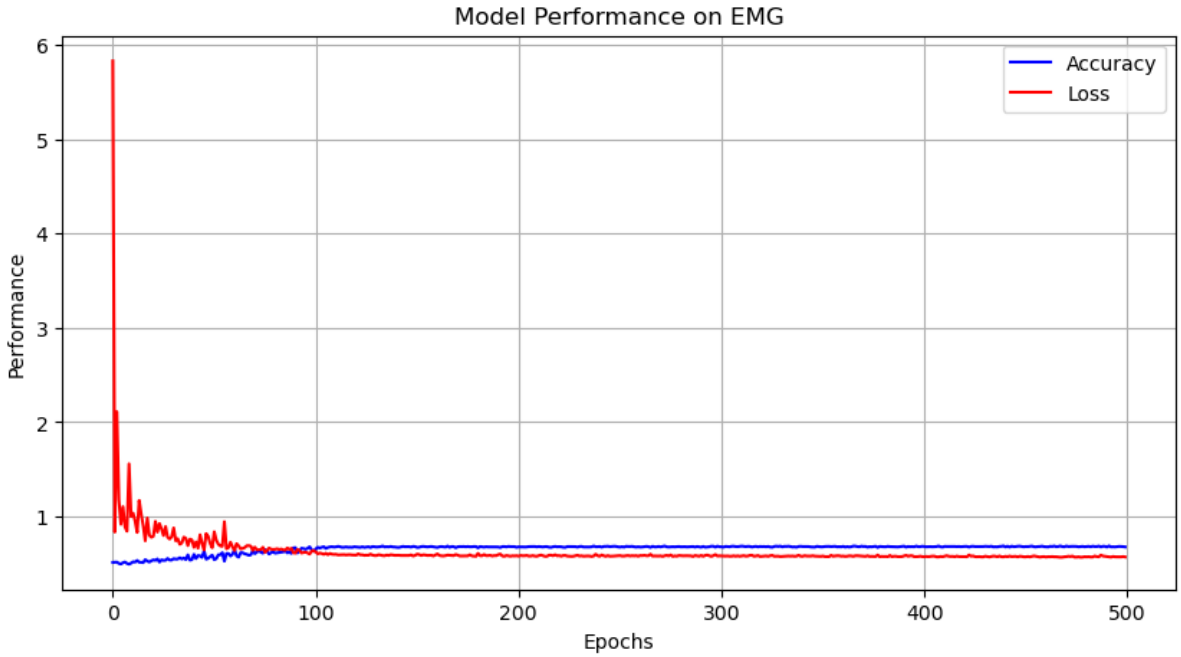
```
import pandas as pd
import matplotlib.pyplot as plt

# Load Data
df = pd.read_csv('history.csv')

# Plot Accuracy
plt.figure(figsize=(10, 5))
plt.plot(df['serial'], df['accuracy'], label='Accuracy', color=
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Model performance for accuracy on emg')
plt.legend()
plt.grid(True)
plt.show()

# Plot Loss
plt.figure(figsize=(10, 5))
plt.plot(df['serial'], df['loss'], label='Loss', color='red')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Model performance for loss on emg')
plt.legend()
plt.grid(True)
plt.show()
```




0.500
0
100
200
300
400
500
Epochs


Model performance for loss on emg
Loss
6
5
4
3
2
1
Loss
0
100
200
300
400
500
Epochs

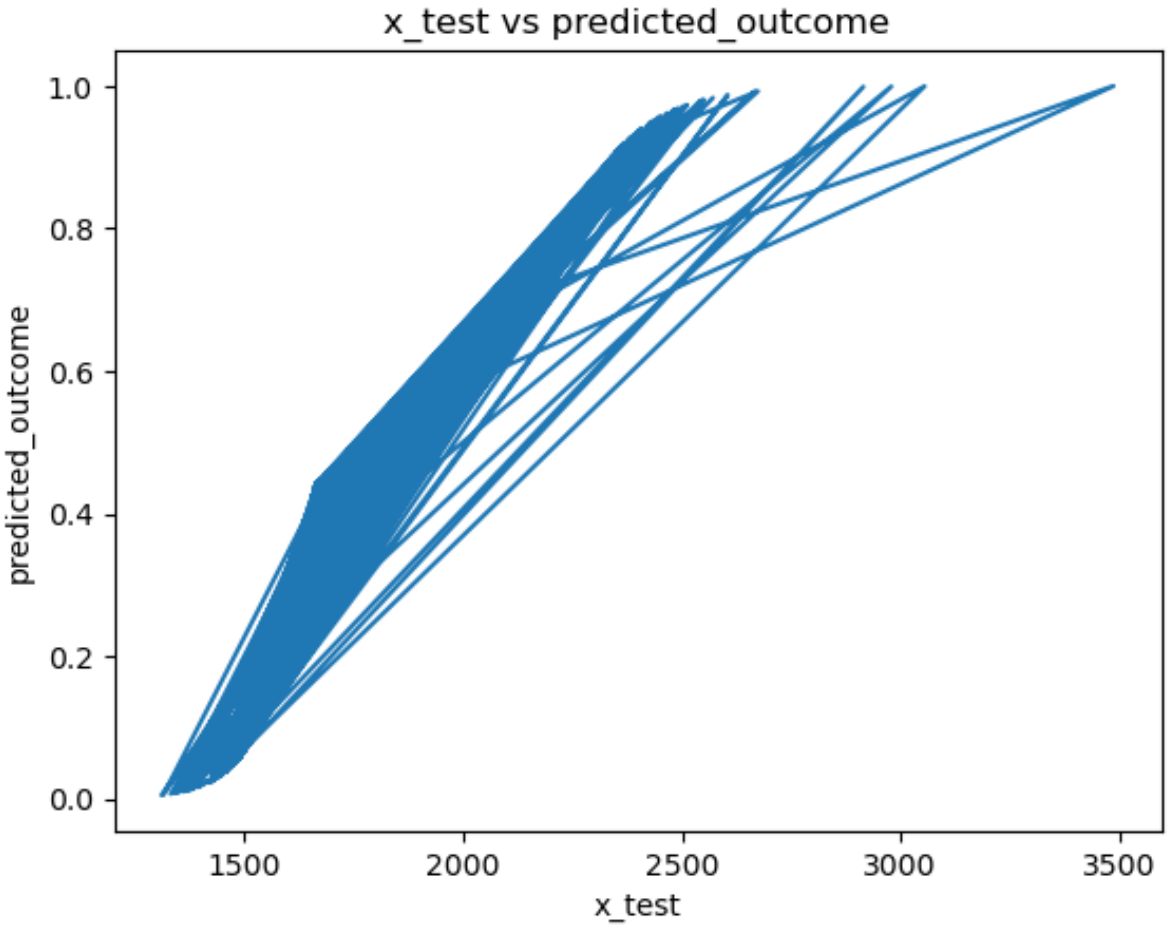
In [8]:
```
import pandas as pd
import matplotlib.pyplot as plt

# Load Data
df = pd.read_csv('history.csv')

# Plot Accuracy and Loss on the same plot
plt.figure(figsize=(10, 5))
plt.plot(df['serial'], df['accuracy'], label='Accuracy', color=
plt.plot(df['serial'], df['loss'], label='Loss', color='red')
plt.xlabel('Epochs')
plt.ylabel('Performance')
plt.title('Model Performance on EMG')
plt.legend()
plt.grid(True)
plt.show()
```



In [25]:
```
predicted_outcome = model.predict(x_test)
#predicted_outcome and x_test save into csv
#predicted_outcome (predicted result vs. x_test)
#print('predicted_outcome', predicted_outcome)

import pandas as pd

# Create a pandas DataFrame with x_test and predicted_outcome
data = {'x_test': x_test.flatten(), 'predicted_outcome': predic
df = pd.DataFrame(data)

# Save the DataFrame to a CSV file
df.to_csv('x_test_predicted_outcome.csv', index=False)
```

35/35 ━━━━━━━━━━━━━━━━━━━━ **0s** 2ms/step

In [26]:
```
#plot outcome (predicted outcome), x_test vs. predicted_outcome
import matplotlib.pyplot as plt

plt.plot(x_test, predicted_outcome)
plt.xlabel('x_test')
plt.ylabel('predicted_outcome')
plt.title('x_test vs predicted_outcome')
plt.show()
```

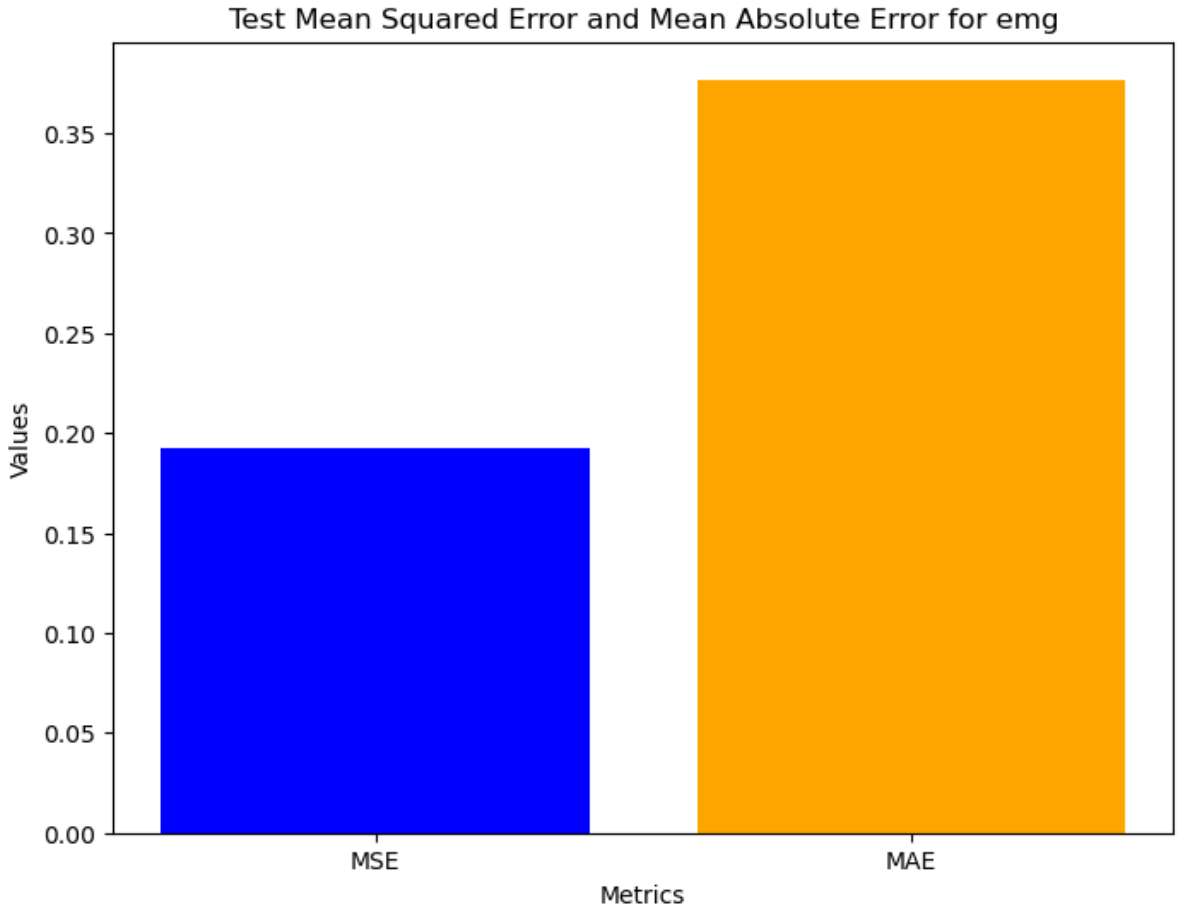


In [27]:

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_e

# Predict outcomes
y_train_pred = model.predict(x_train)
y_test_pred = model.predict(x_test)

# Calculate MSE and MAE for training and testing data
train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)
train_mae = mean_absolute_error(y_train, y_train_pred)
test_mae = mean_absolute_error(y_test, y_test_pred)

print(f"Training mean squared error: {train_mse}")
print(f"Testing mean squared error: {test_mse}")
print(f"Training mean absolute error: {train_mae}")
print(f"Testing mean absolute error: {test_mae}")


```

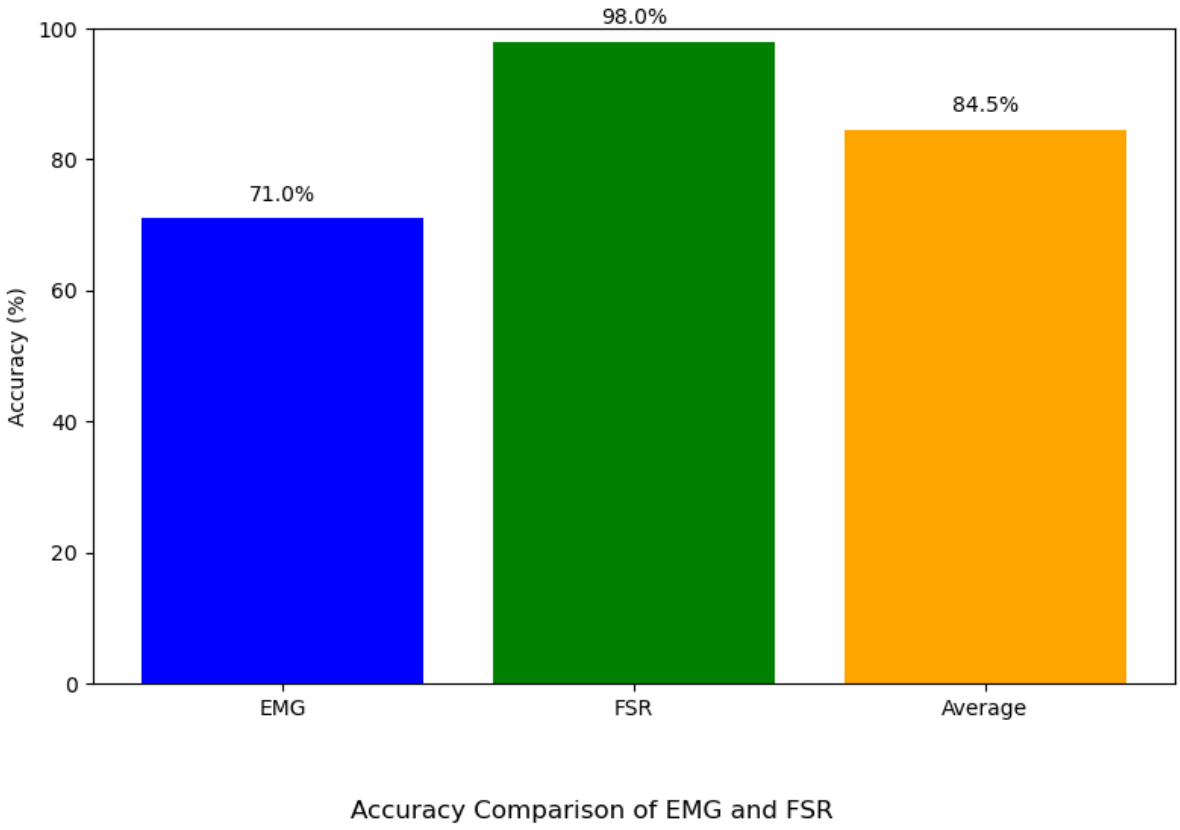
```
138/138 ━━━━━━━━━━━━━━━━━━━━ 0s 511us/step
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 502us/step
Training mean squared error: 0.18852521720404852
Testing mean squared error: 0.1923123210971652
Training mean absolute error: 0.37355227865328994
Testing mean absolute error: 0.3767973965152421
```

In [28]:
```
metrics = ['MSE', 'MAE']
values = [test_mse, test_mae]

plt.figure(figsize=(8, 6))
plt.bar(metrics, values, color=['blue', 'orange'])
plt.xlabel('Metrics')
plt.ylabel('Values')
plt.title('Test Mean Squared Error and Mean Absolute Error for
plt.show()
```



In [34]:
```
import matplotlib.pyplot as plt

# Accuracies
emg_accuracy = 71
fsr_accuracy = 98
average_accuracy = (emg_accuracy + fsr_accuracy) / 2

# Data for plotting
categories = ['EMG', 'FSR', 'Average']
accuracies = [emg_accuracy, fsr_accuracy, average_accuracy]

# Create a bar plot
plt.figure(figsize=(8, 5))
plt.bar(categories, accuracies, color=['blue', 'green', 'orange
plt.ylim(0, 100)

```

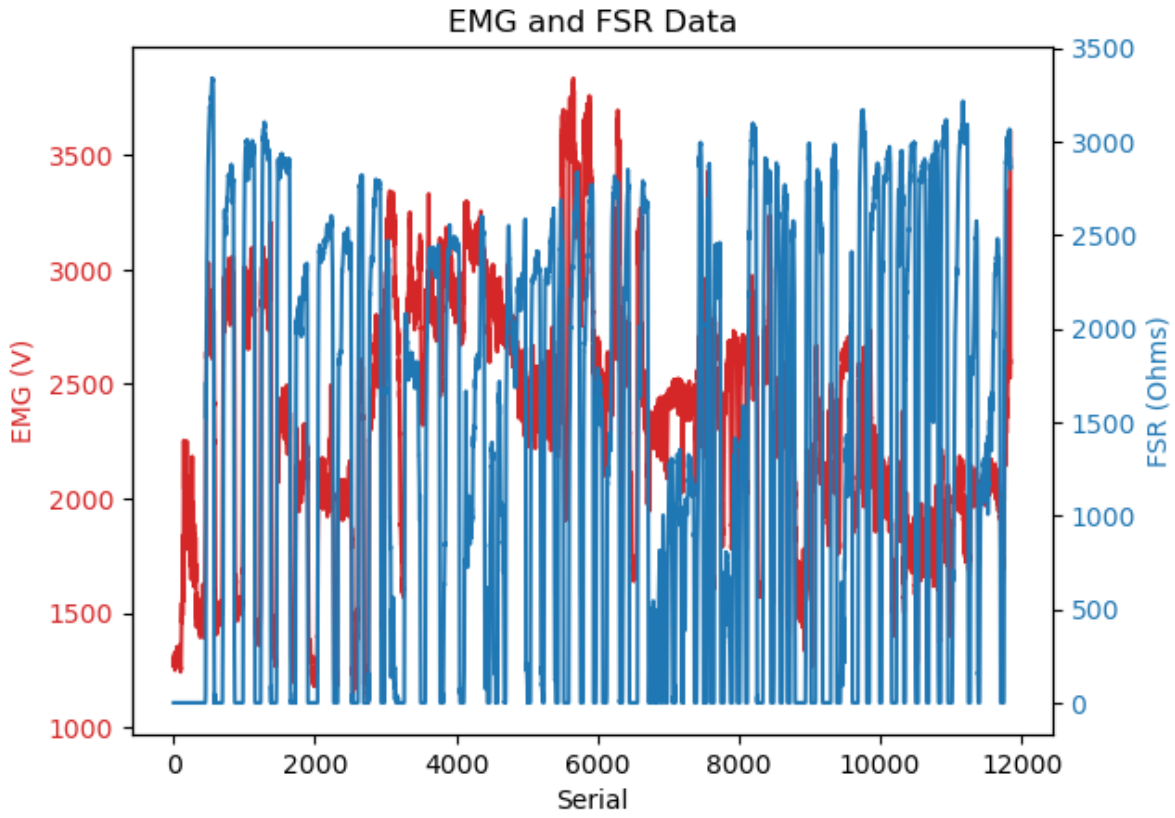
```
# Add titles and labels
plt.ylabel('Accuracy (%)')

# Display the accuracy values on top of the bars
for i, accuracy in enumerate(accuracies):
    plt.text(i, accuracy + 2, f'{accuracy:.1f}%', ha='center',

# Add the title at the bottom
plt.figtext(0.5, -0.1, 'Accuracy Comparison of EMG and FSR', wr

# Adjust layout to make room for the title at the bottom
plt.tight_layout()

# Show the plot
plt.show()
```



In [27]:

```
import pandas as pd
import matplotlib.pyplot as plt

# Read the CSV file
df = pd.read_csv('amna training 1.csv')

# Extract data from DataFrame
serial = df['serial']
emg = df['emg']
fsr = df['fsr']

# Plotting
```

```
fig, ax1 = plt.subplots()

# Plot EMG data
color = 'tab:red'
ax1.set_xlabel('Serial')
ax1.set_ylabel('EMG (V)', color=color)
ax1.plot(serial, emg, color=color)
ax1.tick_params(axis='y', labelcolor=color)

# Create a second y-axis for FSR data
ax2 = ax1.twinx()
color = 'tab:blue'
ax2.set_ylabel('FSR (Ohms)', color=color)
ax2.plot(serial, fsr, color=color)
ax2.tick_params(axis='y', labelcolor=color)

plt.title('EMG and FSR Data')
plt.show()


```



In [ ]:
```
'''from sklearn.preprocessing import StandardScaler
import pandas as pd

# Load CSV data
df = pd.read_csv('Ansa training without headers.csv', header=No

# Display original data
print("Original Data:")
print(df)

# Perform scaling
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)

# Display mean of each feature
print("\nMean of each feature after scaling:")
print(scaler.mean_)

# Display scaled data
print("\nScaled Data:")
print(scaled_data)'''

```

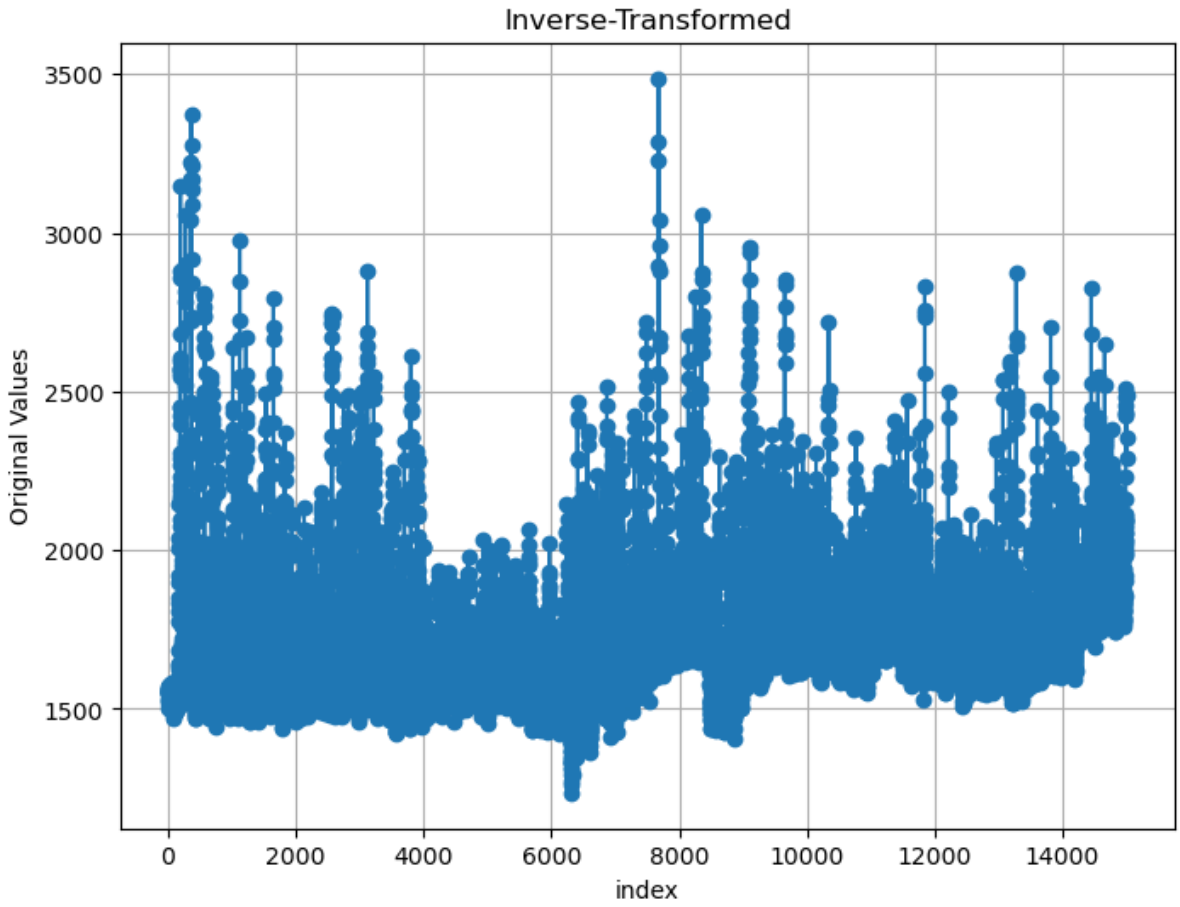
In [ ]:
```
'''# Transform new data
new_data = [[1]]
scaled_new_data = scaler.transform(new_data)

# Display transformed new data
print("\nTransformed new data:")
print(scaled_new_data)'''
```

In [28]:

```
#load StandardScaler
from sklearn.preprocessing import StandardScaler

#load x_train data from CSV file
x_train = pd.read_csv('Ansa training without headers.csv')

#create scaler object
scalery = StandardScaler().fit(x_train)

#transform the x_test data
x_test = pd.read_csv('Ansa training without headers.csv')
x_test = scalery.transform(x_test)

# print transformed x_test
print("this is the scaled array:\n",x_test)

#inverse the x_test data back to original values
x_new = pd.DataFrame(x_test, columns =['x_new'])
x_new_inverse = scalery.inverse_transform(x_new)

# print inversed to original x_test
print("\nthis is the inversed array:\n",x_new_inverse)
```

```
this is the scaled array:
 [[-1.16127221]
 [-0.84097427]
 [-0.92104875]
 ...
 [ 2.55041568]
 [ 2.85658283]
 [ 3.4971787 ]]

this is the inversed array:
 [[1501.]
 [1569.]
 [1552.]
 ...
 [2289.]
 [2354.]
 [2490.]]
```

In [29]:
```
import matplotlib.pyplot as plt

# plot the inverse-transformed x_test data
plt.figure(figsize=(8, 6))
plt.plot(range(len(x_new_inverse)), x_new_inverse, marker='o')
plt.xlabel('index')
plt.ylabel('Original Values')
plt.title('Inverse-Transformed')
plt.grid(True)
plt.show()
```



In [ ]:

In [ ]:

In [ ]: